



IoT end-to-end engineering company

Complete Development from Electronics to Application and Cloud

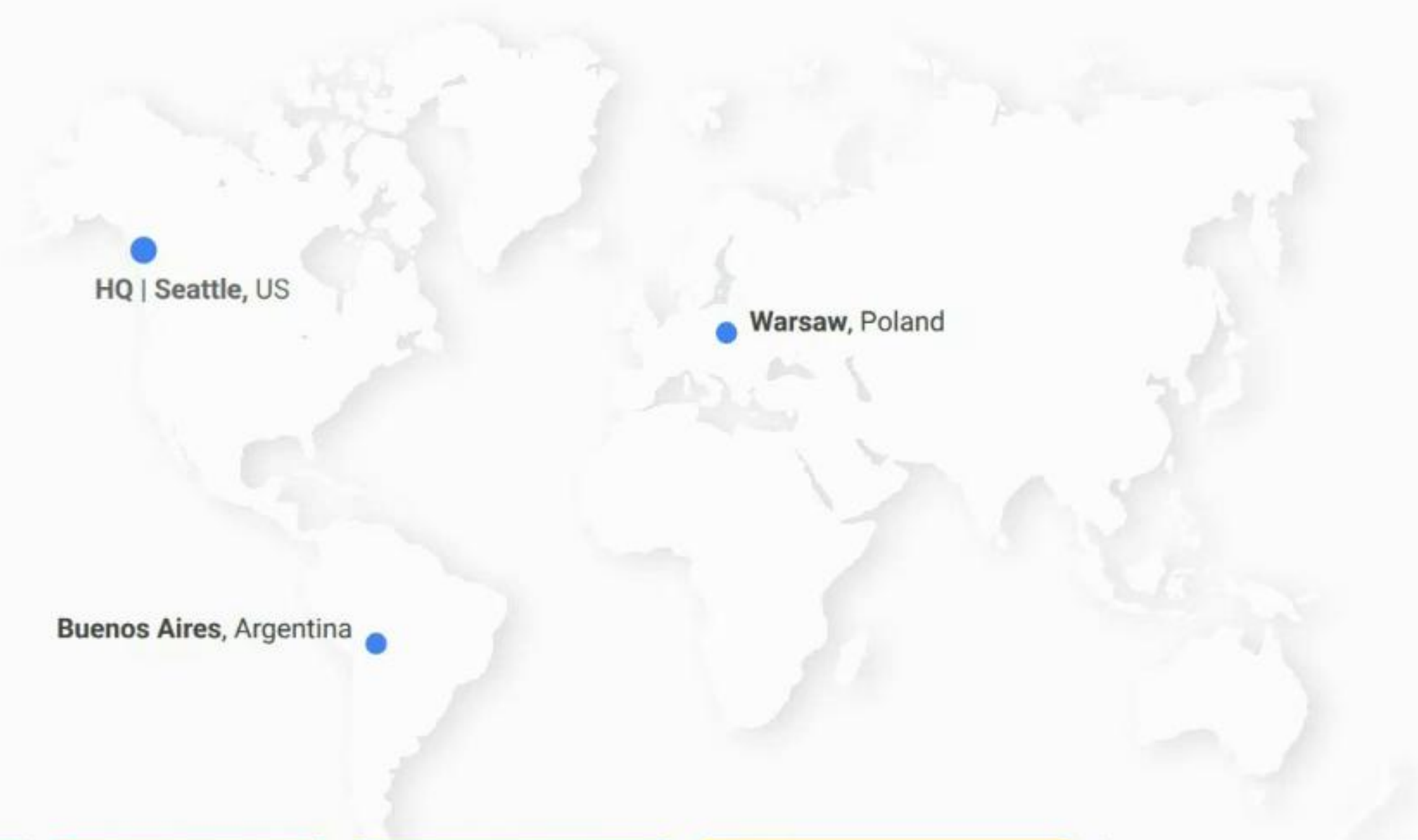


Ecosystem
Vendor





Where we are



HQ | Seattle, US

Warsaw, Poland

Buenos Aires, Argentina

- Software Dev
- Firmware Dev
- Industrial Design
- Electronics Design
- Edge AI / Computer Vision

Welcome to Zephyr RTOS

Zephyr RTOS Course





About us



Dawid Marszalkiewicz

Senior Embedded Developer |
Zephyr RTOS Expert | Technical T...



Dawid Marszalkiewicz



- **Role:** Lead Zephyr Engineer
- **Background:** M.Sc. Embedded Robotics, Wrocław University of Technology
- **Experience:** Senior embedded developer, Ex-CTO
- **Focus:** Zephyr RTOS, FreeRTOS , embedded C/C++
- **Leading:** English group



Mariano Miguel Franceschetti
IOMICO Embedded Software
Engineer



Mariano Franceschetti



- **Role:** Embedded Software Engineer
- **Background:** Electronic Engineer (UNLP)
- **Experience:** Embedded systems & hardware development
- **Focus:** FreeRTOS, Zephyr, PCB design, MCU programming
- **Leading:** Spanish group



Who is this for?



- Basic **C/C++** programming knowledge
- Understanding of **embedded systems** concepts (MCU, memory, peripherals)
- Familiarity with **command line/terminal**
- Basic knowledge of **CMake** and **git**



Format



- Lecture with optional **live demo**
- Homework after each session (checked by us)
- Recordings will be shared
- Questions addressed at the end in a **Q&A session**



Training Path



Course Schedule



	Lecture	English	Spanish
L01	What is Zephyr	Mar 18	Mar 19
L02	West & topologies	Mar 25	Mar 26
L03	App configuration: Kconfig	Apr 1	Apr 2
L04	App configuration: Device tree	Apr 8	Apr 9
	Gap week		
L05	Zephyr custom board	Apr 22	Apr 23
L06	Driver development	Apr 29	Apr 30
L07	Shell integration	May 6	May 7
L08	Unit testing	May 13	May 14



Homeworks



- **Fork** the provided course repository
- Share your fork link with us
- After completing each lecture's tasks — push a tag (tag format will be provided with the task)



Certificate





What is Zephyr?



Zephyr RTOS — At a Glance



- **RTOS for embedded systems**, hosted by the Linux Foundation
- Supports **multiple CPU architectures** (ARM, RISC-V, x86, ARC, Xtensa...)
- **Modular configuration** using Kconfig and Devicetree
- Built-in **driver model** with extensive hardware abstraction
- Backed by an **active open-source community**



Once upon a time...



From Intel Lab Project to Linux Foundation



- Originally developed by **Intel** in **2014**
- Donated to the **Linux Foundation** in **2016**
- At handover: ~7,000 commits, 4 supported platforms:
 - Arduino 101
 - Arduino Due
 - Intel Galileo
 - NXP FRDM-K64F

Source: [The Zephyr Story – Intel](#)



Evolution Highlights

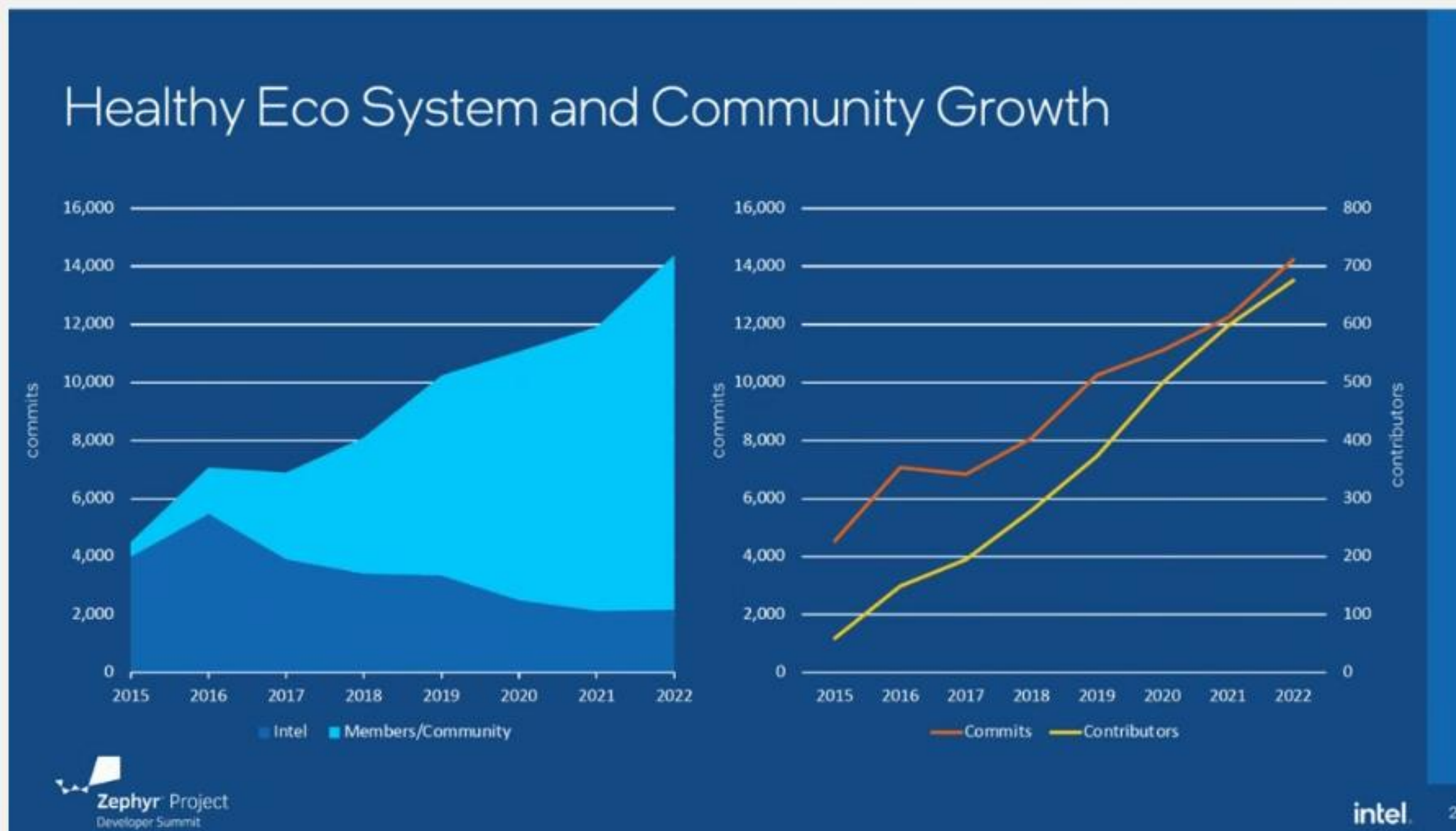


- From ~7K commits → now **80,000+ commits**
- From 4 boards → now over **500+ hardware platforms**
- Support added for **RISC-V, Xtensa, ARC**, more
- Backed by a strong **vendor and contributor community**

Source: [The Zephyr Story — Intel](#)



The Community Behind Zephyr





Zephyr vs FreeRTOS



Philosophy



	FreeRTOS	Zephyr
Model	Lightweight kernel	Full-featured RTOS framework
Philosophy	Minimal core, add libs externally	Everything integrated, modular
Governance	AWS-led	Linux Foundation
Licensing	MIT	Apache 2.0



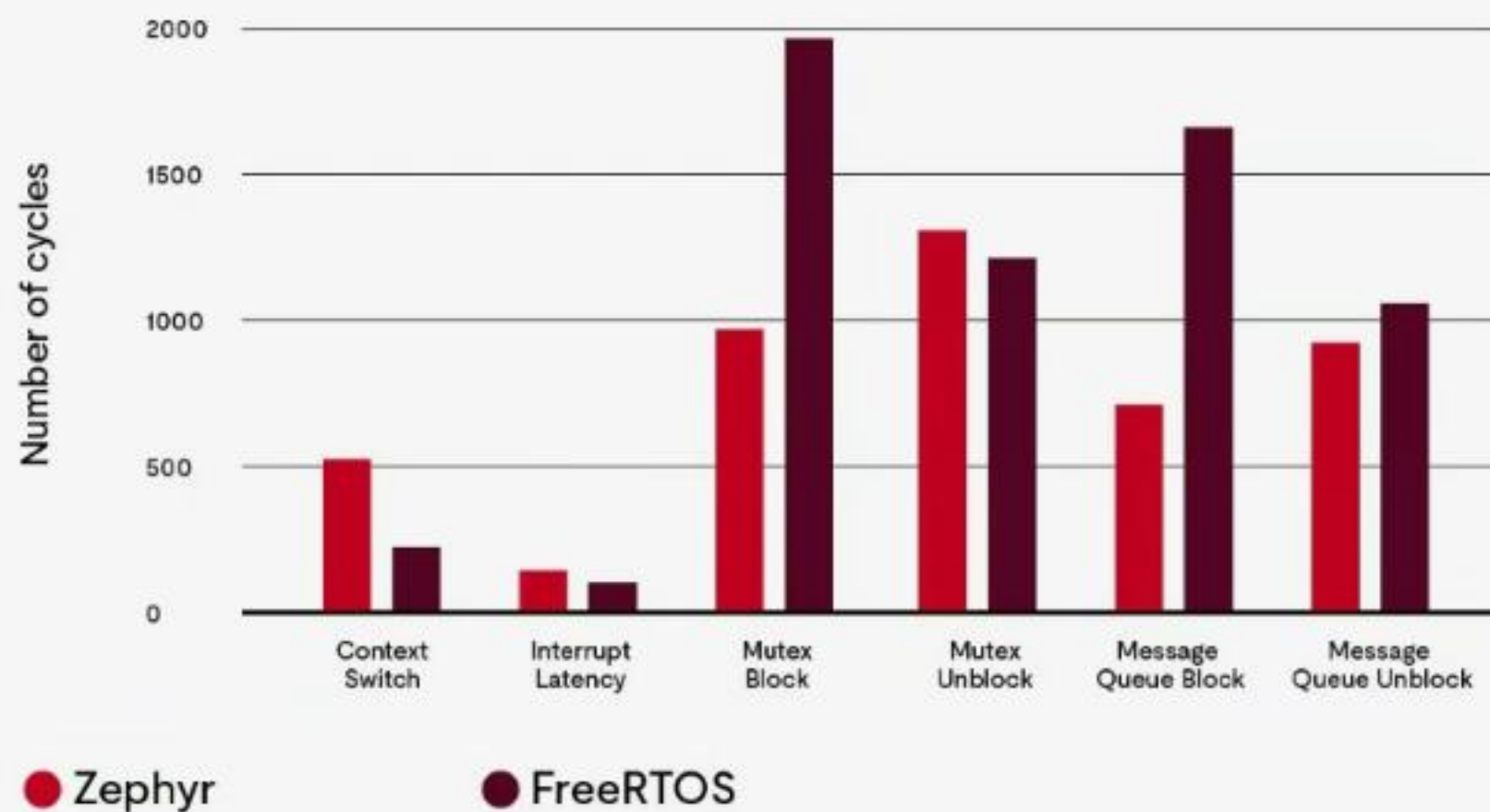
Area	FreeRTOS	Zephyr
Build system	Manual	Unified via <code>west</code>
Configuration	Manual	Kconfig + Devicetree
BSP support	Manual per vendor	Unified + community maintained
Getting started	Very fast	Steeper learning curve
Debug and analysis	Vendor-specific or DIY	Built-in: <code>ram_report</code> , <code>menuconfig</code> ...



Runtime Comparison



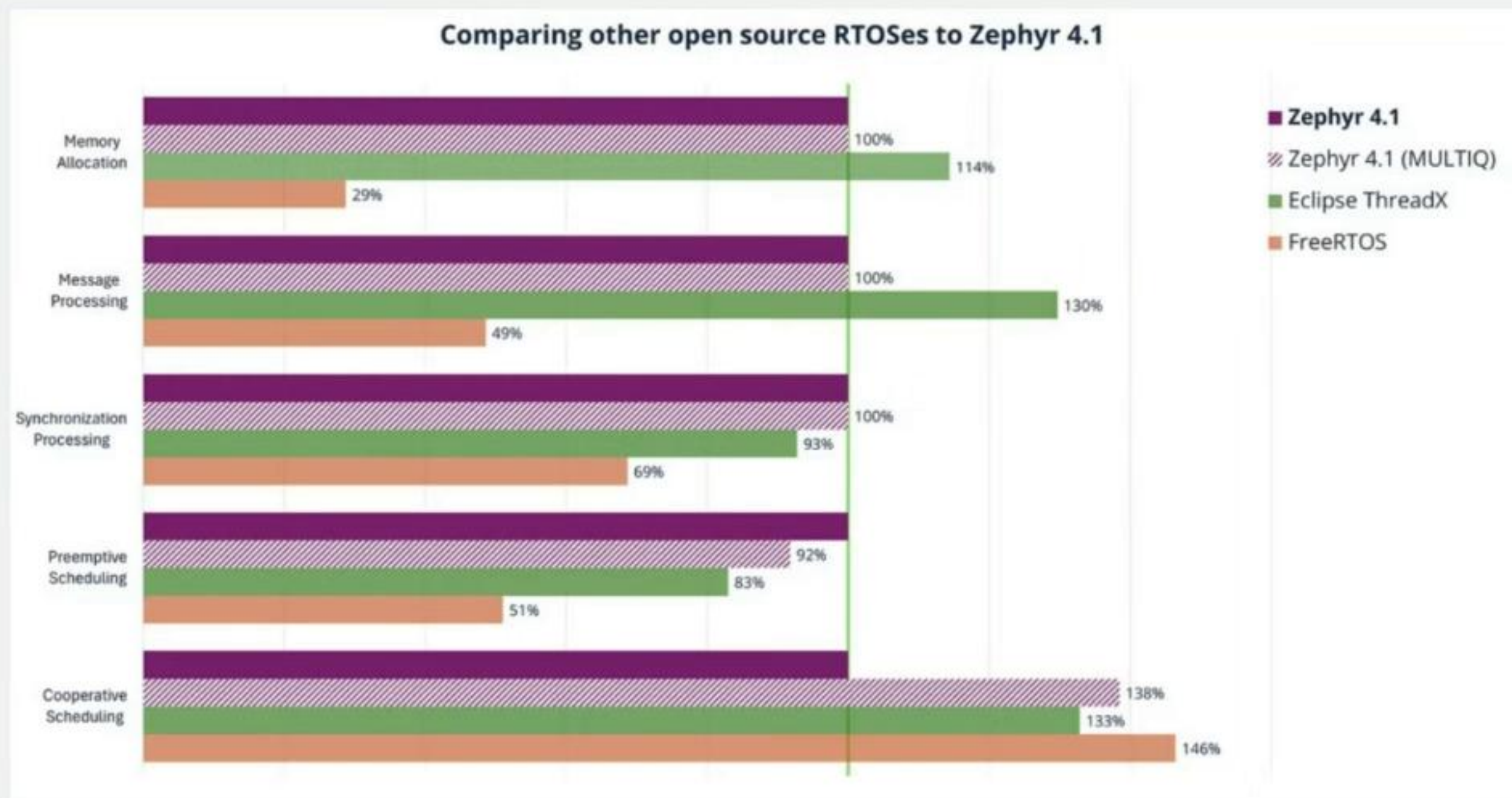
Average execution times of kernel services (1000 measurements)



The average number of cycles of each executed RTOSBench test case.
Each test has been repeated one thousand times.



Runtime Comparison





It's just different...



What's Inside Zephyr?



Devices



- **Drivers** for SPI, I2C, UART, PWM, ADC, etc.
- **Drivers** for displays, sensors, and others
- **USB** device & host stack



Communication



- **TCP/IP**, MQTT, HTTP, CoAP
- **Bluetooth** (BLE & Classic), **CAN**, **LoRa**
- Network modules for **IoT** and industry



3rd Party Integration

- **LVGL** – UI/graphics library
- **Protobuf** – serialization
- **Filesystems** (LittleFS, FAT, NVS)





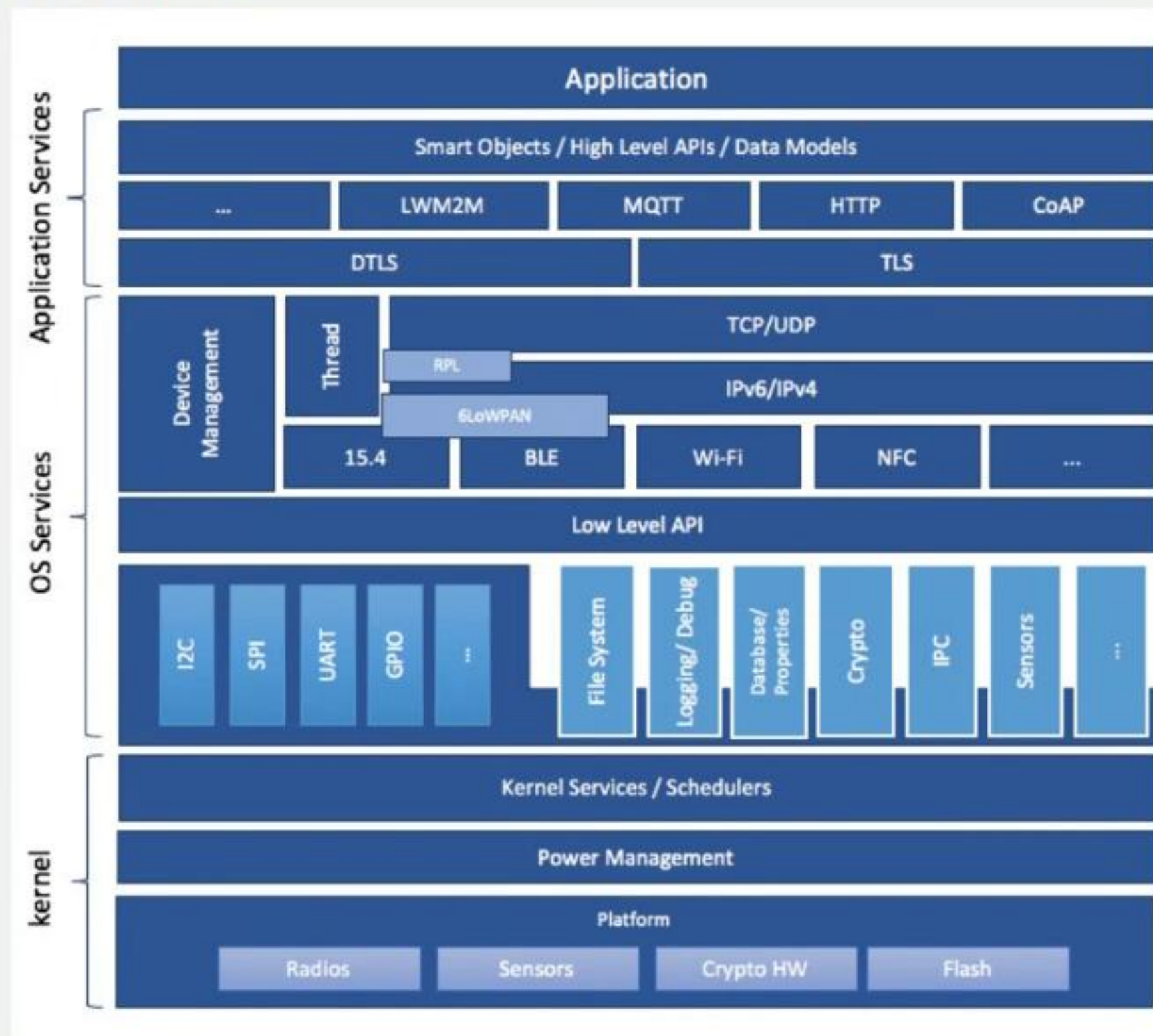
Subsystems and Tooling

- **Logging** system and diagnostics
- **OTA** update module (MCUboot)
- **Testing**, debugging, and analysis tools





Architecture Overview





Getting Started



What Will We Do?



- Walk through the [Zephyr Getting Started Guide](#) together
- Install the Zephyr SDK and dependencies
- Build and flash **Hello World**



Environment Setup Overview



1. Install OS-level dependencies (CMake, Python, DTC, etc.)
2. Create and activate a **Python virtual environment**
3. Install **west** via pip
4. Fetch Zephyr with `west init` + `west update`
5. Install Python requirements from Zephyr
6. Install the **Zephyr SDK**



Building Hello World



```
west build -b <your_board> zephyr/samples/hello_world  
west flash
```

Expected output:

```
*** Booting Zephyr OS build v4.3.0 ***  
Hello World! <your_board>
```



Tasks



0. Read: [Getting Started Guide](#)

1. Set up the development environment

- Install all dependencies and the Zephyr SDK
- Create and activate a Python virtual environment
- Install `west` and fetch Zephyr

2. Build and flash Hello World

- Use `west build -b <board> zephyr/samples/hello_world`
- Try with your hardware board or `native_sim`
- Verify you see the boot banner

Thank You!





Resources



1. [Zephyr Getting Started Guide](#)
2. [The Zephyr Story — Intel](#)
3. [FreeRTOS vs Zephyr Performance Comparison](#)
4. [Zephyr 4.1 Release Notes](#)
5. [Zephyr SDK Toolchain Documentation](#)